

```

        SELECT * from data_lake.accident_data_17_23;
    '''
    cursor.execute(create_table_query)

    # Step 3: Commit the transaction
    conn_psycomp2.commit()
    print("Query executed successfully!")
    columns = cursor.description

    result1 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
    result1 = pd.DataFrame(result1)

```

Query executed successfully!

In [6]: result1.head()

Out[6]:

	Year	Accident severity	municipality	town	First Mode of Transport	Second mode of Transport	Area Type	Light condition	Road Location	Road condition	Road surface	Rc situat
0	2017	Fatal	Breda	BREDA	Car	Pedestrian	Urban area	Darkness	Intersection	Wet/ damp	Brick	Be
1	2017	Fatal	Breda	BREDA	Lorry	Other	Urban area	Daylight	Intersection	Wet/ damp	Brick	Intersect - 4 ar
2	2017	Fatal	Breda	BREDA	Lorry	Other	Urban area	Daylight	Road section	Dry	Asphalt (other)	Strai rc
3	2017	Injured	Breda	BAVEL	Car	Lorry	Rural area	Darkness	Road section	Wet/ damp	Asphalt (other)	Be
4	2017	Injured	Breda	BAVEL	Car	Other	Rural area	Darkness	Road section	Wet/ damp	Porous asphalt	Strai rc

In [7]: result1['town'].unique()

Out[7]: array(['BREDA', 'BAVEL', 'PRINSENBEK', 'TETERINGEN', 'ULVENHOUT',
'GALDER', 'ETTEN-LEUR'], dtype=object)

```

In [8]: # Step 2: Execute SQL command to create the table
create_table_query = '''
        SELECT * from data_lake.safe_driving;

```

```

    ...
    cursor.execute(create_table_query)

    # Step 3: Commit the transaction
    conn_psycomp2.commit()
    print("Query executed successfully!")
    columns = cursor.description

    result2 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
    result2 = pd.DataFrame(result2)

```

Query executed successfully!

In []:

In [9]: result2.head()

Out[9]:

	eventid	event_start	event_end	duration_seconds	latitude	longitude	speed_kmh	end_speed_kmh	maxwaa
0	40547819	2019-12-04 10:23:42.500	2019-12-04 10:23:43.800	1.3	51.601670	4.790512	20.921473	25.749504	0.753
1	40967778	2019-12-04 14:52:26.500	2019-12-04 14:52:27.700	1.2	51.601450	4.800820	27.216045	31.777254	0.780
2	40961813	2019-12-04 16:49:02.300	2019-12-04 16:49:02.700	0.4	51.600636	4.810158	69.300120	68.328120	0.000
3	40126170	2019-12-04 07:29:34.500	2019-12-04 07:29:42.500	8.0	51.574230	4.809726	41.842945	41.842945	41.842
4	41007863	2019-12-04 17:11:38.500	2019-12-04 17:11:42.100	3.6	51.577084	4.755841	3.996007	2.160004	1.419

Filtering from Safe Driving Data

Only is_valid=True incidents are selected, because the definition from ANWB says so. Refer to Data Sourcing under project brief

Plan is to filter the incidents which count for greater than 4000

Other incidents are too small to be considered, so only top 5 incidents will be used in the analysis moving forward

These incidents are - Harsh Cornering (HC1 and HC2) , Speed (SP1 and SP2) , Braking (HB1)

```
In [54]: # Step 2: Execute SQL command to create the table
# Method 1: Psycpg2

conn_psycpg2 = psycpg2.connect(**db_params)
# Step 1: Create a cursor object
cursor = conn_psycpg2.cursor()
create_table_query = '''
    SELECT TRIM(category) as category,
    trim(incident_severity) as incident_severity,
    COUNT(*) as tot_count from data_lake.safe_driving
    where is_valid='True'
    -- In SQL you can use 1 for column you use. You are using numbers just to avoid typing the entire c
    group by 1,2
    ORDER BY tot_count desc;
'''
cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycpg2.commit()
print("Query executed successfully!")
columns = cursor.description

result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
result3 = pd.DataFrame(result3)

conn_psycpg2.close()

result3
```

Query executed successfully!

Out[54]:

	category	incident_severity	tot_count
0	HARSH CORNERING	HC1	396023
1	SPEED	SP1	356892
2	SPEED	SP2	27289
3	BRAKING	HB1	26740
4	HARSH CORNERING	HC2	4047
5	ACCELERATING	HA1	3546
6	SPEED	SP3	2382
7	BRAKING	HB2	1777
8	HARSH CORNERING	HC4	285
9	ACCELERATING	HA2	285
10	HARSH CORNERING	HC13	276
11	HARSH CORNERING	HC3	224
12	SPEED	SP4	202
13	BRAKING	HB3	143
14	HARSH CORNERING	HC16	54
15	HARSH CORNERING	HC5	51
16	HARSH CORNERING	HC14	43
17	HARSH CORNERING	HC7	41
18	ACCELERATING	HA3	36
19	SPEED	SP5	26
20	HARSH CORNERING	HC17	10
21	HARSH CORNERING	HC19	6
22	HARSH CORNERING	HC8	5

	category	incident_severity	tot_count
23	HARSH CORNERING	HC6	4
24	HARSH CORNERING	HC15	2
25	HARSH CORNERING	HC18	2
26	HARSH CORNERING	HC21	1

In EDA, always validate your numbers. The count shown below will be matched in the newly created table

```
In [61]: 396023+356892+27289+26740+4047
```

```
Out[61]: 810991
```

Created a temporary table for purposes

From previous analysis, only the mentioned incidents are selected, and we are ignoring all other incidents as they are all irrelevant mathematically

These incidents are - Harsh Cornering (HC1 and HC2) , Speed (SP1 and SP2) , Braking (HB1)

```
In [59]: # Step 2: Execute SQL command to create the table
# Method 1: Psycopg2

conn_psycopg2 = psycopg2.connect(**db_params)
# Step 1: Create a cursor object
cursor = conn_psycopg2.cursor()
create_table_query = '''

    -- Always include drop statement before creating your custom tables
    drop table data_lake.filtered_base_data;
    create table data_lake.filtered_base_data as (
    SELECT * from data_lake.safe_driving
    where is_valid='True'

    -- Including only incidents which we want to use from the previous analysis
    and trim(incident_severity) in ('HC1','HC2','SP1','SP2','HB1')
```

```

        );

'''
cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycomp2.commit()
print("Query executed successfully!")
columns = cursor.description

#result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
#result3 = pd.DataFrame(result3)

conn_psycomp2.close()

#result3

```

Query executed successfully!

Newly created table matches the count that we did during our EDA

```

In [60]: # Step 2: Execute SQL command to create the table
# Method 1: Psycopg2

conn_psycomp2 = psycopg2.connect(**db_params)
# Step 1: Create a cursor object
cursor = conn_psycomp2.cursor()
create_table_query = '''
    select count(*) from data_lake.filtered_base_data;
'''
cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycomp2.commit()
print("Query executed successfully!")
columns = cursor.description

result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
result3 = pd.DataFrame(result3)

conn_psycomp2.close()

```

```
result3
```

Query executed successfully!

```
Out[60]:      count
0  810991
```

Now we have to match the data with our accident table

```
In [63]: # Step 2: Execute SQL command to create the table
# Method 1: Psycopg2

conn_psycopg2 = psycopg2.connect(**db_params)
# Step 1: Create a cursor object
cursor = conn_psycopg2.cursor()
create_table_query = '''
    select * from data_lake.accident_data_17_23 limit 10;
'''
cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycopg2.commit()
print("Query executed successfully!")
columns = cursor.description

result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
result3 = pd.DataFrame(result3)

conn_psycopg2.close()

result3
```

Query executed successfully!

Out[63]:

	Year	Accident severity	municipality	town	First Mode of Transport	Second mode of Transport	Area Type	Light condition	Road Location	Road condition	Road surface	s
0	2017	Fatal	Breda	BREDA	Car	Pedestrian	Urban area	Darkness	Intersection	Wet/damp	Brick	
1	2017	Fatal	Breda	BREDA	Lorry	Other	Urban area	Daylight	Intersection	Wet/damp	Brick	Inte-
2	2017	Fatal	Breda	BREDA	Lorry	Other	Urban area	Daylight	Road section	Dry	Asphalt (other)	
3	2017	Injured	Breda	BAVEL	Car	Lorry	Rural area	Darkness	Road section	Wet/damp	Asphalt (other)	
4	2017	Injured	Breda	BAVEL	Car	Other	Rural area	Darkness	Road section	Wet/damp	Porous asphalt	
5	2017	Injured	Breda	BAVEL	Delivery van	Car	Rural area	Darkness	Road section	Wet/damp	Asphalt (other)	
6	2017	Injured	Breda	BREDA	Bicycle	Motorcycle	Urban area	Daylight	Intersection	Dry	Asphalt (other)	Inte-
7	2017	Injured	Breda	BREDA	Bicycle	Other	Urban area	Daylight	Intersection	Dry	Asphalt (other)	Inte-
8	2017	Injured	Breda	BREDA	Bicycle	Other	Urban area	Daylight	Intersection	Dry	Brick	
9	2017	Injured	Breda	BREDA	Bicycle	Other	Unknown	Darkness	Road section	Unknown	Unknown	U

Only common columns between filtered_base_data and the accident data columns are Year (Column name - Year in accident data, and event_start in ANWB data) and road name (Column name - street in accident data, and road_name in ANWB data)

We will try to join only road_name to see whether there are any common places in both the data

```
In [5]: # Step 2: Execute SQL command to create the table
# Method 1: Psycpg2
```



```
conn_psycpg2 = psycpg2.connect(**db_params)
# Step 1: Create a cursor object
cursor = conn_psycpg2.cursor()
create_table_query = '''

-- I have used complex sub queries, but I have commented wherever possible
-- write to me or ping me if you have queries, or if you find any issues in the query

select
    TRIM(category) as category,
        trim(incident_severity) as incident_severity,
        COUNT(*) as tot_count
from
    data_lake.filtered_base_data a1

join

-- By using this join, the operation will be much faster, since we are technically filtering only data from
(

    -- I have used the results as the "b" table temporarily to save space.
    -- Below sub query will find the matching street names from the two tables

    select anwb_road_name
    from
        ( -- Sub Query to find the unique road names from the anwb data
            select lower(road_name) anwb_road_name
            from
                data_lake.filtered_base_data
            group by 1
        ) a
    where a.anwb_road_name in
        (
            -- Sub Query to find the unique road names from the new accident data
            select lower(street) new_accident_street
            from
                data_lake.accident_data_17_23
            group by 1
        )
)b
```

```

-- joined on the road name with the larger base table to see the number of rows matching
-- see the results and the subsequent calculation to see whether our analysis is valid

on lower(a1.road_name)=b.anwb_road_name
group by 1,2
;
...
cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycpg2.commit()
print("Query executed successfully!")
columns = cursor.description

result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
result3 = pd.DataFrame(result3)

conn_psycpg2.close()

result3

```

Query executed successfully!

Out[5]:

	category	incident_severity	tot_count
0	SPEED	SP1	349587
1	HARSH CORNERING	HC1	387624
2	BRAKING	HB1	25582
3	SPEED	SP2	26831
4	HARSH CORNERING	HC2	3936

In [6]: 349587+387624+25582+26831+3936

Out[6]: 793560

Rule of Thumb!! Always validate your data to be a good analytics Translator or a Data Engineer! you have lost around 2% of data. i.e, after joining on the street name, we have reduced rows on our base table from 810991 to 793560 (~17k rows) . You should always account for lost data anywhere in your analysis.

The loss is fine, because we are losing 2% data, and in any business, there is acceptable level of error. If you lose more

than 15% data, then we should further deep dive to see the root cause. For now, continue with the analysis

```
In [7]: # Step 2: Execute SQL command to create the table
# Method 1: Psycpg2

conn_psycpg2 = psycpg2.connect(**db_params)
# Step 1: Create a cursor object
cursor = conn_psycpg2.cursor()
create_table_query = '''
    select * from data_lake.accident_data_17_23 limit 10;
'''
cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycpg2.commit()
print("Query executed successfully!")
columns = cursor.description

result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
result3 = pd.DataFrame(result3)

conn_psycpg2.close()

result3
```

Query executed successfully!

Out [7]:

	Year	Accident severity	municipality	town	First Mode of Transport	Second mode of Transport	Area Type	Light condition	Road Location	Road condition	Road surface	s
0	2017	Fatal	Breda	BREDA	Car	Pedestrian	Urban area	Darkness	Intersection	Wet/damp	Brick	
1	2017	Fatal	Breda	BREDA	Lorry	Other	Urban area	Daylight	Intersection	Wet/damp	Brick	Inter-
2	2017	Fatal	Breda	BREDA	Lorry	Other	Urban area	Daylight	Road section	Dry	Asphalt (other)	
3	2017	Injured	Breda	BAVEL	Car	Lorry	Rural area	Darkness	Road section	Wet/damp	Asphalt (other)	
4	2017	Injured	Breda	BAVEL	Car	Other	Rural area	Darkness	Road section	Wet/damp	Porous asphalt	
5	2017	Injured	Breda	BAVEL	Delivery van	Car	Rural area	Darkness	Road section	Wet/damp	Asphalt (other)	
6	2017	Injured	Breda	BREDA	Bicycle	Motorcycle	Urban area	Daylight	Intersection	Dry	Asphalt (other)	Inter-
7	2017	Injured	Breda	BREDA	Bicycle	Other	Urban area	Daylight	Intersection	Dry	Asphalt (other)	Inter-
8	2017	Injured	Breda	BREDA	Bicycle	Other	Urban area	Daylight	Intersection	Dry	Brick	
9	2017	Injured	Breda	BREDA	Bicycle	Other	Unknown	Darkness	Road section	Unknown	Unknown	U

Checking minimum and maximum available date in anwb data

```
In [13]: # Step 2: Execute SQL command to create the table
# Method 1: Psycpg2

conn_psycpg2 = psycpg2.connect(**db_params)
# Step 1: Create a cursor object
cursor = conn_psycpg2.cursor()
create_table_query = '''
    select min(DATE_TRUNC('day',event_start)) min_date,
```

```

        max(DATE_TRUNC('day',event_start)) max_date
    from
    data_lake.filtered_base_data
    limit 10;
'''
cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycopg2.commit()
print("Query executed successfully!")
columns = cursor.description

result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
result3 = pd.DataFrame(result3)

conn_psycopg2.close()

result3

```

Query executed successfully!

```

Out[13]:      min_date  max_date
0  2018-01-01  2024-02-29

```

Checking minimum and maximum available year in accident data

```

In [21]: # Step 2: Execute SQL command to create the table
# Method 1: Psycopg2

conn_psycopg2 = psycopg2.connect(**db_params)
# Step 1: Create a cursor object
cursor = conn_psycopg2.cursor()
create_table_query = '''
    -- Year is the reserved keyword, I guess?!
    -- extract year is the function used to find year from a date, but for a column name, this is weird
    -- I have not used postgresql, but I'm familiar with generic SQL.
    select min("Year") as min_year
    , max("Year") as max_year
    from
    data_lake.accident_data_17_23
    limit 10;
'''

```

```

'''
cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycomp2.commit()
print("Query executed successfully!")
columns = cursor.description

result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
result3 = pd.DataFrame(result3)

conn_psycomp2.close()

result3

```

Query executed successfully!

```

Out[21]:   min_year  max_year
0      2017      2023

```

Now we have data from 2018 to 2023. Some points to note

Since we have only 2 months in 2024 data available to us, we can use 2023 data for data imputation purposes

Carefully follow the below cells to see how I have added the data

```

In [28]: # Step 2: Execute SQL command to create the table
# Method 1: Psycpg2

conn_psycomp2 = psycpg2.connect(**db_params)
# Step 1: Create a cursor object
cursor = conn_psycomp2.cursor()
create_table_query = '''
    -- extract year is the function used to find year from a date, but for a column name, this is weird
    -- In Other SQL implementations, I have used it fine, but here, I have to adapt to certain constraints
    select "Year" as year
    , "Accident severity" as severity
    , sum(accidents)
    from
    data_lake.accident_data_17_23
    where "Year" = 2018

```

```

        group by 1,2;
    '''
cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycomp2.commit()
print("Query executed successfully!")
columns = cursor.description

result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
result3 = pd.DataFrame(result3)

conn_psycomp2.close()

result3

```

Query executed successfully!

Out[28]:

	year	severity	sum
0	2018	Fatal	10
1	2018	Injured	178
2	2018	Material Damage Only	1550

In [103]: *# Step 2: Execute SQL command to create the table*
Method 1: Psycopg2

```

conn_psycomp2 = psycopg2.connect(**db_params)
# Step 1: Create a cursor object
cursor = conn_psycomp2.cursor()
create_table_query = '''
    select
        extract(year from event_start) as year
        ,incident_severity
        ,count(*)
    from
        data_lake.filtered_base_data
    where
        DATE_TRUNC('day',event_start) between '2018-01-01' and '2018-12-31'

    group by 1,2

```

```

    ... ;
cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycoPg2.commit()
print("Query executed successfully!")
columns = cursor.description

result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
result3 = pd.DataFrame(result3)

conn_psycoPg2.close()

result3

```

Query executed successfully!

```

Out[103]:
   year  incident_severity  count
0  2018                  HB1   4051
1  2018                  HC1  44705
2  2018                  HC2    242
3  2018                  SP1  50827
4  2018                  SP2   3675

```

HB1 probability calculation for Fatal accident

```
In [46]: 100 * (10/4051)
```

```
Out[46]: 0.24685262898049865
```

HB1 probability calculation for Fatal accident

```
In [47]: 100 * (10/242)
```

```
Out[47]: 4.132231404958678
```


In this process we are merely dividing two numbers. the result provides the probability that, out of given incidents, how many can be accidents?

Now, take 2018 for example. HB1 - Harsh breaking incidents are 4051. In 2018, we have 10 fatal accidents. Now, we can calculate the probability of HB1 being a fatal accident.

look at the above calculation. 10 fatal accidents over the year / 4051 incidents over the year. this means, every HB1 incident has 0.2% chance of becoming a fatal accident.

if the numbers are higher, for example, HC1 - Harsh cornering 1, then the fatal accident probability will be lower. i.e, $100 * (10/44705) = 0.02\%$ chance of becoming a fatal accident

if the numbers are lower, for example, HC2 - Harsh cornering 2, then the fatal accident probability will be higher. i.e, $100 * (10/242) = 4.1\%$ chance of becoming a fatal accident.

This also aligns with ANWB data definition. i.e, severity of the incident is higher with a higher number

Creating final Table Joining accident data and ANWB data

Things to consider

We have matched the street names

However, we did not address the year column. i.e, we have the year level data in accident table, date level data in anwb data

So we should distribute the match the incidents with accidents

How are we planning to do this?

One way to do is, divide the number of accidents for each date, and distribute the accident types as new columns

Rewriting column names for accident data table, for writing queries faster

```
In [55]: # Step 2: Execute SQL command to create the table
# Method 1: Psycopg2

conn_psycopg2 = psycopg2.connect(**db_params)
# Step 1: Create a cursor object
```

```
cursor = conn_psycpg2.cursor()
create_table_query = '''
drop table data_lake.accident_data_renamed;
create table data_lake.accident_data_renamed as
(
select
"Year" as year,
"Accident severity" as accident_severity,
municipality,
town,
"First Mode of Transport" as first_mode_of_transport,
"Second mode of Transport" as second_mode_of_transport,
"Area Type" as area_type,
"Light condition" as light_condition,
"Road condition" as road_condition,
"Road surface" as road_surface,
"Road situation" as road_situation,
"Speed limit" as speed_limit,
street,
weather,
accidents
from
data_lake.accident_data_17_23
);
'''
cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycpg2.commit()
print("Query executed successfully!")
#columns = cursor.description

#result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
#result3 = pd.DataFrame(result3)

conn_psycpg2.close()
```

Query executed successfully!

```
In [96]: # Step 2: Execute SQL command to create the table
# Method 1: Psycpg2
```

```
conn_psycopg2 = psycopg2.connect(**db_params)
# Step 1: Create a cursor object
cursor = conn_psycopg2.cursor()
create_table_query = '''
select * from data_lake.accident_data_renamed limit 10;    '''
cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycopg2.commit()
print("Query executed successfully!")
columns = cursor.description

result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
result3 = pd.DataFrame(result3)

conn_psycopg2.close()
result3
```

Query executed successfully!

Out[96]:

	year	accident_severity	municipality	town	first_mode_of_transport	second_mode_of_transport	area_type	light_c
0	2017	Fatal	Breda	BREDA	Car	Pedestrian	Urban area	☐
1	2017	Fatal	Breda	BREDA	Lorry	Other	Urban area	
2	2017	Fatal	Breda	BREDA	Lorry	Other	Urban area	
3	2017	Injured	Breda	BAVEL	Car	Lorry	Rural area	☐
4	2017	Injured	Breda	BAVEL	Car	Other	Rural area	☐
5	2017	Injured	Breda	BAVEL	Delivery van	Car	Rural area	☐
6	2017	Injured	Breda	BREDA	Bicycle	Motorcycle	Urban area	
7	2017	Injured	Breda	BREDA	Bicycle	Other	Urban area	
8	2017	Injured	Breda	BREDA	Bicycle	Other	Urban area	
9	2017	Injured	Breda	BREDA	Bicycle	Other	Unknown	☐

Calculating Probability through SQL

Notice that some probabilities are much higher 640% in certain conditions, but these are due to less number of incidents captured by ANWB than the actual accidents captured by SWOV. These are typically material damage accidents, and involve other modes of transport such as lorry too.

We calculate probability to assign a value to mention that under specific conditions we are expecting an event at the highest probability.

Of all accidents recorded, we have material damage accidents to be higher than the recorded incidents. So 5 such probabilities are greater than 100.

For students -> You should impute this value, as typically the percentage is between 0 and 100, and not exceeding 100

```
In [101]: # Step 2: Execute SQL command to create the table
# Method 1: Psycpg2

conn_psycpg2 = psycpg2.connect(**db_params)
# Step 1: Create a cursor object
cursor = conn_psycpg2.cursor()
create_table_query = '''
select
a.incident_year
,a.incident_severity
,a.incident_cnt
,b.accident_severity
,b.accident_cnt
,100 * (b.accident_cnt::float /a.incident_cnt::float ) as accident_probability
from

( select
extract(year from event_start) as incident_year
,trim(incident_severity) as incident_severity
,count(*) incident_cnt
from
data_lake.filtered_base_data
group by 1,2
) a

join

(
select year as accident_year
,accident_severity
,sum(accidents) accident_cnt
from data_lake.accident_data_renamed
where year>=2018
group by 1,2
order by 1,2
) b
on a.incident_year=b.accident_year
where (100 * (b.accident_cnt::float /a.incident_cnt::float ))>100
-- and trim(incident_severity)='HB1'
-- Numbering makes much more sense!!
order by 1,2,3,4,5
;
```

```

'''
cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycopg2.commit()
print("Query executed successfully!")
columns = cursor.description

result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
result3 = pd.DataFrame(result3)

conn_psycopg2.close()

result3

```

Query executed successfully!

```
Out[101]:
```

	incident_year	incident_severity	incident_cnt	accident_severity	accident_cnt	accident_probability
0	2018	HC2	242	Material Damage Only	1550	640.495868
1	2019	HC2	556	Material Damage Only	1656	297.841727
2	2020	HC2	482	Material Damage Only	1135	235.477178
3	2021	HC2	733	Material Damage Only	1283	175.034106
4	2022	HC2	973	Material Damage Only	1406	144.501542

Creating the table with calculated probabilities

```

In [109]: # Step 2: Execute SQL command to create the table
# Method 1: Psycopg2

conn_psycopg2 = psycopg2.connect(**db_params)
# Step 1: Create a cursor object
cursor = conn_psycopg2.cursor()
create_table_query = '''
drop table data_lake.accident_probability;
create table data_lake.accident_probability as
(
select

```

```

a.incident_year
,a.incident_severity
,a.incident_cnt
,b.accident_severity
,b.accident_cnt
,100 * (b.accident_cnt::float /a.incident_cnt::float ) as accident_probability
from

( select
extract(year from event_start) as incident_year
,trim(incident_severity) as incident_severity
,count(*) incident_cnt
from
data_lake.filtered_base_data
group by 1,2
) a

join

(
select year as accident_year
,accident_severity
,sum(accidents) accident_cnt
from data_lake.accident_data_renamed
where year>=2018
group by 1,2
order by 1,2
) b
on a.incident_year=b.accident_year

);

...
cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycomp2.commit()
print("Query executed successfully!")
columns = cursor.description

#result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
#result3 = pd.DataFrame(result3)

```

```
conn_psycpg2.close()
```

Query executed successfully!

```
In [141]: # Step 2: Execute SQL command to create the table
# Method 1: Psycpg2

conn_psycpg2 = psycpg2.connect(**db_params)
# Step 1: Create a cursor object
cursor = conn_psycpg2.cursor()
create_table_query = '''
select * from data_lake.accident_probability where incident_year=2018 and incident_severity='HC1';
'''
cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycpg2.commit()
print("Query executed successfully!")
columns = cursor.description

result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
result3 = pd.DataFrame(result3)

conn_psycpg2.close()

result3
```

Query executed successfully!

```
Out[141]:
```

	incident_year	incident_severity	incident_cnt	accident_severity	accident_cnt	accident_probability
0	2018	HC1	44705	Material Damage Only	1550	3.467174
1	2018	HC1	44705	Injured	178	0.398166
2	2018	HC1	44705	Fatal	10	0.022369

```
In [126]: # Step 2: Execute SQL command to create the table
# Method 1: Psycpg2

conn_psycpg2 = psycpg2.connect(**db_params)
# Step 1: Create a cursor object
```



```
cursor = conn_psycopg2.cursor()
create_table_query = '''
select
*
from
data_lake.filtered_base_data
limit 10'''
cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycopg2.commit()
print("Query executed successfully!")
columns = cursor.description

result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
result3 = pd.DataFrame(result3)

conn_psycopg2.close()

result3
```

Query executed successfully!

Out[126...]	eventid	event_start	event_end	duration_seconds	latitude	longitude	speed_kmh	end_speed_kmh	maxwaarc
0	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	1.2	51.59696	4.791709	17.702784	27.358849	0.75317
1	19010480	2018-12-09 10:48:19.800	2018-12-09 10:48:21.900	2.1	51.59887	4.791461	17.702784	16.093440	0.90441
2	19008704	2018-12-09 10:25:49.000	2018-12-09 10:25:55.000	6.0	51.57767	4.797670	69.201790	62.764416	69.20179
3	19024309	2018-12-09 13:00:24.000	2018-12-09 13:00:32.000	8.0	51.57212	4.764968	62.764416	64.373760	64.37376
4	19034873	2018-12-09 18:28:28.100	2018-12-09 18:28:29.200	1.1	51.59943	4.749152	28.968191	28.968191	0.72903
5	19018928	2018-12-09 12:14:10.200	2018-12-09 12:14:12.400	2.2	51.57235	4.768282	33.796223	32.186880	0.77891
6	19028916	2018-12-09 14:00:53.000	2018-12-09 14:01:00.000	7.0	51.59026	4.803701	41.842945	41.842945	41.84294
7	19016108	2018-12-09 12:26:06.900	2018-12-09 12:26:08.300	1.4	51.58640	4.796607	22.530815	25.749504	0.77409
8	19039532	2018-12-09 15:25:46.000	2018-12-09 15:25:52.000	6.0	51.58886	4.771495	43.452290	41.842945	45.06163
9	19034794	2018-12-09 18:34:03.000	2018-12-09 18:34:15.000	12.0	51.57476	4.801606	41.842945	41.842945	43.45229

In [170...

Query executed successfully!

Out[170...

	count
0	2100337

In [150...

```
# Step 2: Execute SQL command to create the table
# Method 1: Psycopg2

conn_psycopg2 = psycopg2.connect(**db_params)
```

```
# Step 1: Create a cursor object
cursor = conn_psycopg2.cursor()
create_table_query = '''
select * from
(
    select eventid
    ,event_start
    ,event_end
    ,duration_seconds
    ,latitude
    ,longitude
    ,speed_kmh
    ,end_speed_kmh
    ,maxwaarde
    ,category
    ,a.incident_severity
    ,b.accident_severity
    ,is_valid
    ,road_segment_id
    ,road_manager_type
    ,road_number
    ,road_name
    ,place_name
    ,municipality_name
    ,road_manager_name
    ,b.accident_probability
    from
    data_lake.filtered_base_data a
    left join data_lake.accident_probability b
    on extract(year from a.event_start) = b.incident_year
    and trim(a.incident_severity)= b.incident_severity
) a
where eventid=19010481
limit 10;
'''

cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycopg2.commit()
print("Query executed successfully!")
columns = cursor.description
```

```

result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
result3 = pd.DataFrame(result3)

conn_psycopg2.close()

result3

```

Query executed successfully!

```

Out[150]:

```

	eventid	event_start	event_end	duration_seconds	latitude	longitude	speed_kmh	end_speed_kmh	maxwaard
0	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	1.2	51.59696	4.791709	17.702784	27.358849	0.75317
1	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	1.2	51.59696	4.791709	17.702784	27.358849	0.75317
2	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	1.2	51.59696	4.791709	17.702784	27.358849	0.75317

3 rows × 21 columns

```

In [156]: # Step 2: Execute SQL command to create the table
# Method 1: Psycopg2

conn_psycopg2 = psycopg2.connect(**db_params)
# Step 1: Create a cursor object
cursor = conn_psycopg2.cursor()
create_table_query = '''
select * from data_lake.accident_data_renamed
where year = 2018
--and accident_severity='Fatal'
and street='Doornboslaan'
limit 10;
'''

cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycopg2.commit()
print("Query executed successfully!")
columns = cursor.description

result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]

```

```
result3 = pd.DataFrame(result3)

conn_psycpg2.close()

result3
```

Query executed successfully!

```
Out[156...   year  accident_severity  municipality  town  first_mode_of_transport  second_mode_of_transport  area_type  light_co
```

	year	accident_severity	municipality	town	first_mode_of_transport	second_mode_of_transport	area_type	light_co
0	2018	Fatal	Breda	BREDA	Motorcycle	Other	Urban area	[
1	2018	Injured	Breda	BREDA	Moped	Bicycle	Urban area	
2	2018	Injured	Breda	BREDA	Moped	Moped	Urban area	
3	2018	Material Damage Only	Breda	BREDA	Moped	Car	Unknown	
4	2018	Material Damage Only	Breda	BREDA	Motorcycle	Car	Urban area	
5	2018	Material Damage Only	Breda	BREDA	Car	Pedestrian	Urban area	[
6	2018	Material Damage Only	Breda	BREDA	Car	Bicycle	Urban area	
7	2018	Material Damage Only	Breda	BREDA	Car	Bicycle	Urban area	[
8	2018	Material Damage Only	Breda	BREDA	Car	Light-moped	Urban area	
9	2018	Material Damage Only	Breda	BREDA	Car	Moped	Urban area	

```
In [133... # Step 2: Execute SQL command to create the table
# Method 1: Psycpg2

conn_psycpg2 = psycpg2.connect(**db_params)
# Step 1: Create a cursor object
```

```

cursor = conn_psycopg2.cursor()
create_table_query = '''
select count(*) from
data_lake.filtered_base_data
limit 10;
'''

cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycopg2.commit()
print("Query executed successfully!")
columns = cursor.description

result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
result3 = pd.DataFrame(result3)

conn_psycopg2.close()

result3

```

Query executed successfully!

```

Out[133]:      count
0    810991

```

Final Merged Query

```

In [179]: # Step 2: Execute SQL command to create the table
# Method 1: Psycopg2

conn_psycopg2 = psycopg2.connect(**db_params)
# Step 1: Create a cursor object
cursor = conn_psycopg2.cursor()
create_table_query = '''
drop table data_lake.accident_incident_merged_table;
create table data_lake.accident_incident_merged_table
as
(
    select eventid
    ,event_start
    ,event_end

```

```
,duration_seconds
,latitude
,longitude
,speed_kmh
,end_speed_kmh
,maxwaarde
,category
,a.incident_severity
,b.accident_severity
,is_valid
,road_segment_id
,road_manager_type
,road_number
,road_name
,place_name
,municipality_name
,road_manager_name
,b.accident_probability
,c.first_mode_of_transport
,c.second_mode_of_transport
,c.area_type
,c.light_condition
,c.road_condition
,c.road_surface
,c.road_situation
,c.speed_limit
,c.street
,c.weather
from
data_lake.filtered_base_data a
left join data_lake.accident_probability b
on extract(year from a.event_start) = b.incident_year
and trim(a.incident_severity)= b.incident_severity
left join data_lake.accident_data_renamed c
on lower(a.road_name)=lower(c.street)
and b.accident_severity=c.accident_severity
and extract(year from a.event_start)=c.year

);
...
cursor.execute(create_table_query)
```

```

# Step 3: Commit the transaction
conn_psycopg2.commit()
print("Query executed successfully!")
#columns = cursor.description

#result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
#result3 = pd.DataFrame(result3)

conn_psycopg2.close()

result3

```

Query executed successfully!

Out[179]...

accident_severity

0	Material Damage Only
1	Injured
2	Fatal

Verify at which level the data is created

In [186]...

```

# Step 2: Execute SQL command to create the table
# Method 1: Psycopg2

conn_psycopg2 = psycopg2.connect(**db_params)
# Step 1: Create a cursor object
cursor = conn_psycopg2.cursor()
create_table_query = '''
Select
eventid
,event_start
,event_end
,incident_severity
,accident_severity
,accident_probability
,first_mode_of_transport
,second_mode_of_transport
,speed_limit
,street

```



```
,weather
from
data_lake.accident_incident_merged_table
Where eventid=19010481
;
...
cursor.execute(create_table_query)

# Step 3: Commit the transaction
conn_psycopg2.commit()
print("Query executed successfully!")
columns = cursor.description

result3 = [{columns[index][0]: column for index, column in enumerate(value)} for value in cursor.fetchall()]
result3 = pd.DataFrame(result3)

conn_psycopg2.close()

result3
```

Query executed successfully!

Out [186...

	eventid	event_start	event_end	incident_severity	accident_severity	accident_probability	first_mode_of_transp
0	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	HC1	Fatal	0.022369	Motorcycl
1	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	HC1	Injured	0.398166	Mop
2	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	HC1	Injured	0.398166	Mop
3	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	HC1	Material Damage Only	3.467174	Oth
4	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	HC1	Material Damage Only	3.467174	(
5	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	HC1	Material Damage Only	3.467174	(
6	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	HC1	Material Damage Only	3.467174	(
7	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	HC1	Material Damage Only	3.467174	(
8	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	HC1	Material Damage Only	3.467174	(
9	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	HC1	Material Damage Only	3.467174	(
10	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	HC1	Material Damage Only	3.467174	
11	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	HC1	Material Damage Only	3.467174	Motorcycl
12	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	HC1	Material Damage Only	3.467174	Mop
13	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	HC1	Material Damage Only	3.467174	(
14	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	HC1	Material Damage Only	3.467174	(

	eventid	event_start	event_end	incident_severity	accident_severity	accident_probability	first_mode_of_transp
15	19010481	2018-12-09 10:48:51.300	2018-12-09 10:48:52.500	HC1	Material Damage Only	3.467174	(

Now we have all of the data in one single table. The way to read table is as follows:

The event id 19010481 happened in 2018, the exact date is provided by ANWB

From the data of SWOV, we can infer that there 0.022% chance of being involved in an accident if you travel by motorcycle on Doornboslaan, under dry conditions

If you travel by car, there is high chance of 3.4% of being involved in an incident that includes material damage only.

Based on this data, you can start building the model.

Do not create more tables, because you can create views which is lightweight on the server. This example is just for you to understand how to connect data from two different data sources. This tutorial is only intended if you are planning to use newly added accident data (accident_data_17_23).